Navigation2 Single-Point Navigation

> [!IMPORTANT]
>
> **Note:** This course requires at least one grid map for navigation. If you have not built a map, please refer to the previous SLAM mapping course to build one first.

1. Course Content

Basic:

- Run the sample program, and a map will be loaded in RViz. In the RViz interface, use the "2D Goal Pose" tool to give the car a target point. The car will plan a path based on its environment and move to the destination. If it encounters obstacles along the way, it will autonomously avoid them and stop after reaching the destination.

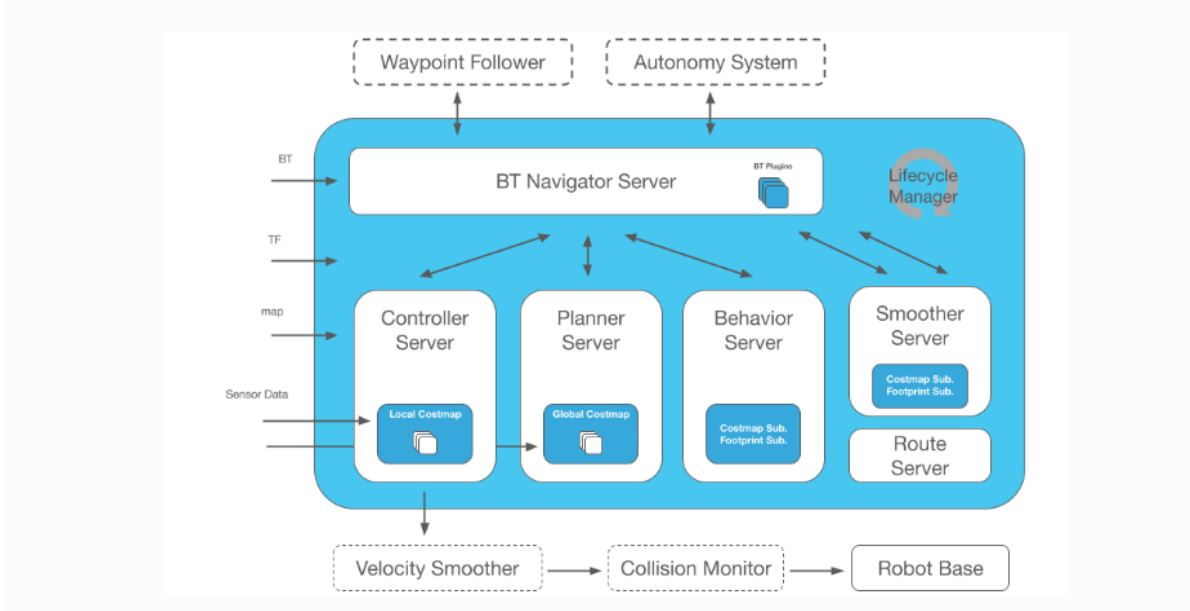
Advanced:

- Understand the concepts of global and local costmaps, debugging navigation speed and obstacle avoidance effects, and the principles of the AMCL localization algorithm.

> [!TIP]
>
> - The navigation function supports a shortcut launch command. If you have already built a grid map named `map`, you can start the navigation function with the following shortcut:
>
> ```
> nav2
> ```

2. Navigation2 Introduction

- Navigation2 Overall Architecture Diagram



Navigation2 includes the following tools:

- Tools for loading, providing, and storing maps (Map Server)
- Tools for localizing the robot on the map (AMCL)
- Path planning tools to move from point A to point B while avoiding obstacles (Nav2 Planner)
- Tools for controlling the robot while following a path (Nav2 Controller)
- Tools for converting sensor data into a costmap representation of the robot's world (Nav2 Costmap 2D)
- Tools for building complex robot behaviors using behavior trees (Nav2 Behavior Trees and BT Navigator)
- Tools for calculating recovery behaviors in case of failure (Nav2 Recoveries)
- Tools for following a sequence of waypoints (Nav2 Waypoint Follower)
- Tools and watchdogs for managing server lifecycles (Nav2 Lifecycle Manager)
- Plugins for enabling user-defined algorithms and behaviors (Nav2 Core)

Navigation 2 (Nav2) is the navigation framework included in ROS 2, designed to enable a mobile robot to move from point A to point B in a safe manner. Therefore, Nav2 can perform dynamic path planning, calculate motor speeds, avoid obstacles, and execute recovery behaviors.

Nav2 uses Behavior Trees (BT) to call modular servers to perform actions. Actions can be calculating a path, control efforts, recovery, or other navigation-related actions. These actions are independent nodes that communicate with the Behavior Tree via action servers.

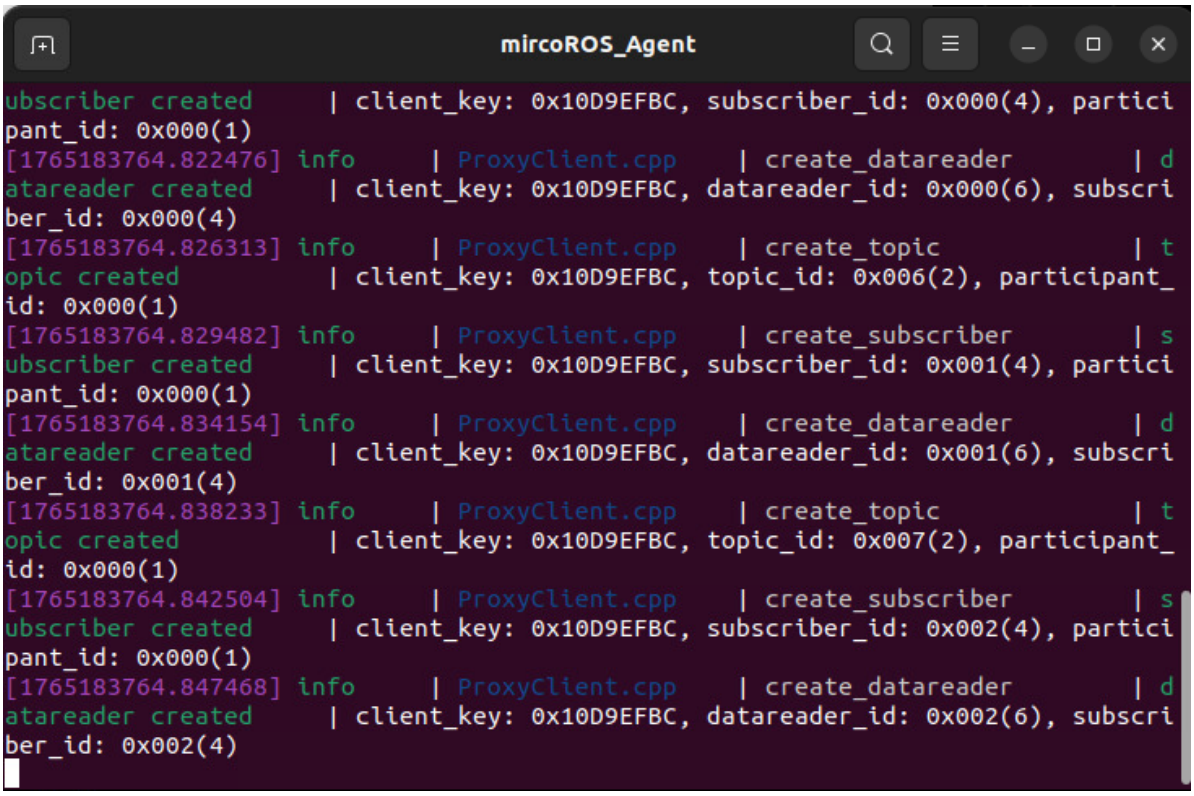
3. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:



4. Run the Example

4.1 Single-Point Navigation Function

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

(ORIN board)

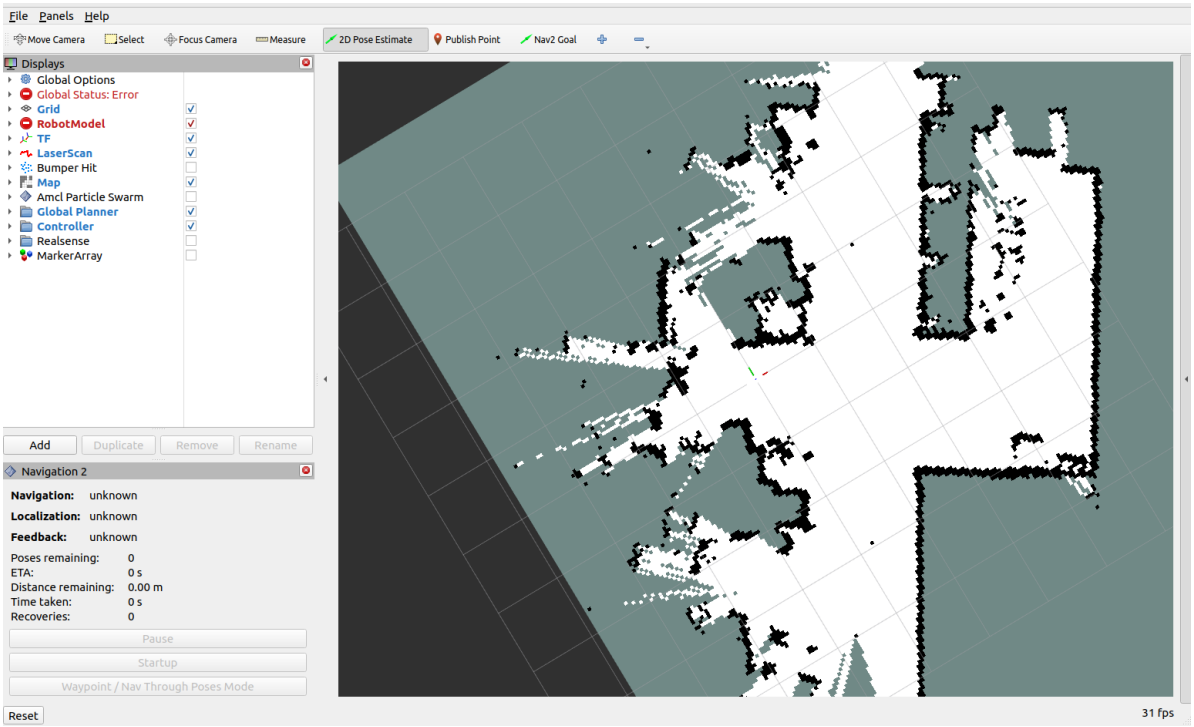
```
ros2 launch m3_bringup nav2.launch.py
```

(RDK & PI5 boards) Run in separate steps, with visualization launched on the accompanying virtual machine `Rosmaster-M3-Ubuntu22.04`,

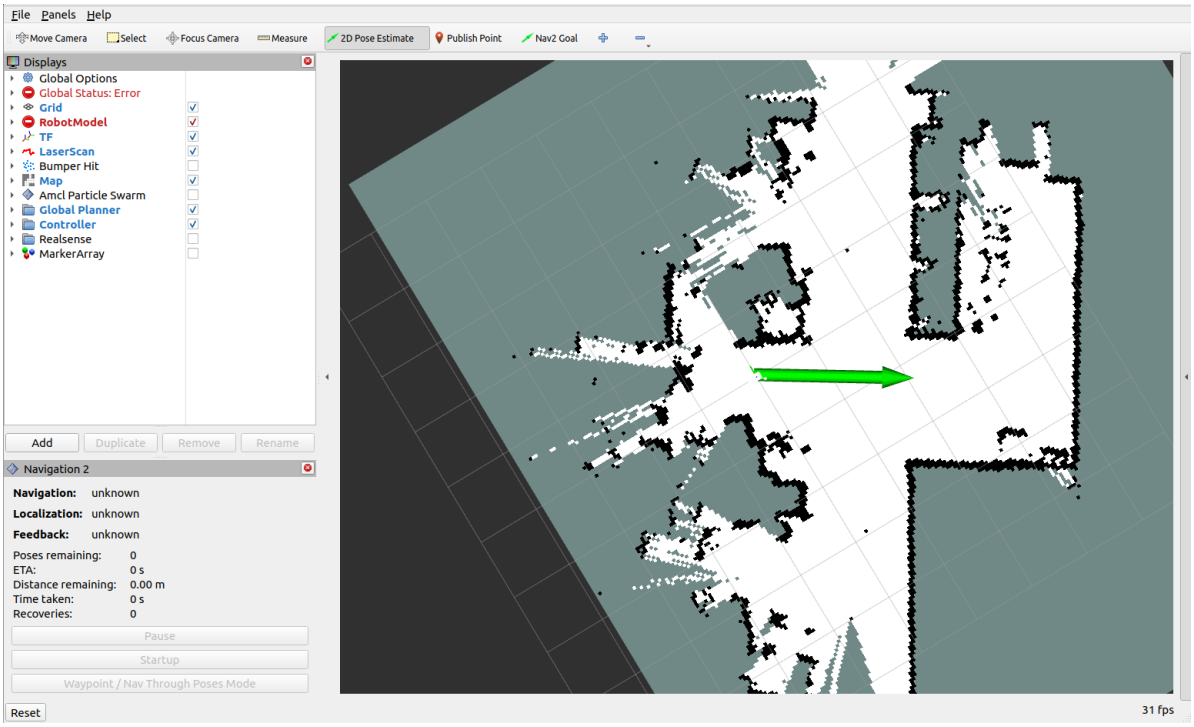
```
# On the virtual machine
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/nav2.rviz
# On the host machine
ros2 launch m3_bringup nav2.launch.py gui:=False
```

> [!NOTE]
>
> **Optional Launch Parameter Descriptions:**
>
> - `map`

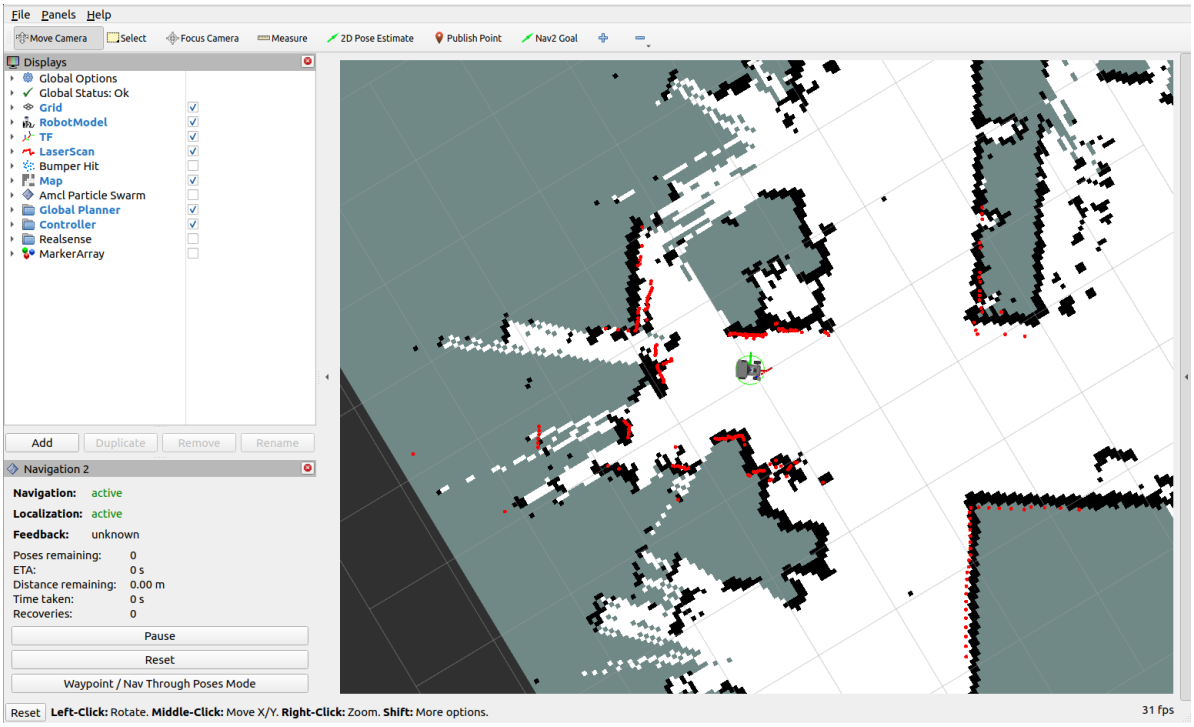
    - The name of the map, default is `map.yaml`. The map must be saved in the `~/map` directory on the system.
- `gui`
    - Whether to start the RViz display window, default is `True`.
- `pose_map`
    - The name of the posegraph map. If `slam_toolbox` is chosen as the localization method for navigation, the pose map name must be provided. The default is `map`. Only the name needs to be provided, without the suffix, and it will be automatically searched for in the `~/map` directory.
- `load_state_filename`
    - The name of the pbstream pose map. If `cartographer` is chosen as the localization method, this must be provided. The default is `map.pbstream`.
- `params_file`
    - The navigation parameter file, default is `~/yahboomM3_ws/user_ws/src/m3_bringup/config/navigation_config/M3_nav2_params.yaml`.
- `localization_mode`
    - The localization method, default is `amcl`. `cartographer`, `slam_toolbox`, and `rtabmap` are also available.
- `slam`
    - Whether to start synchronous mapping and navigation mode, performing mapping and navigation simultaneously.



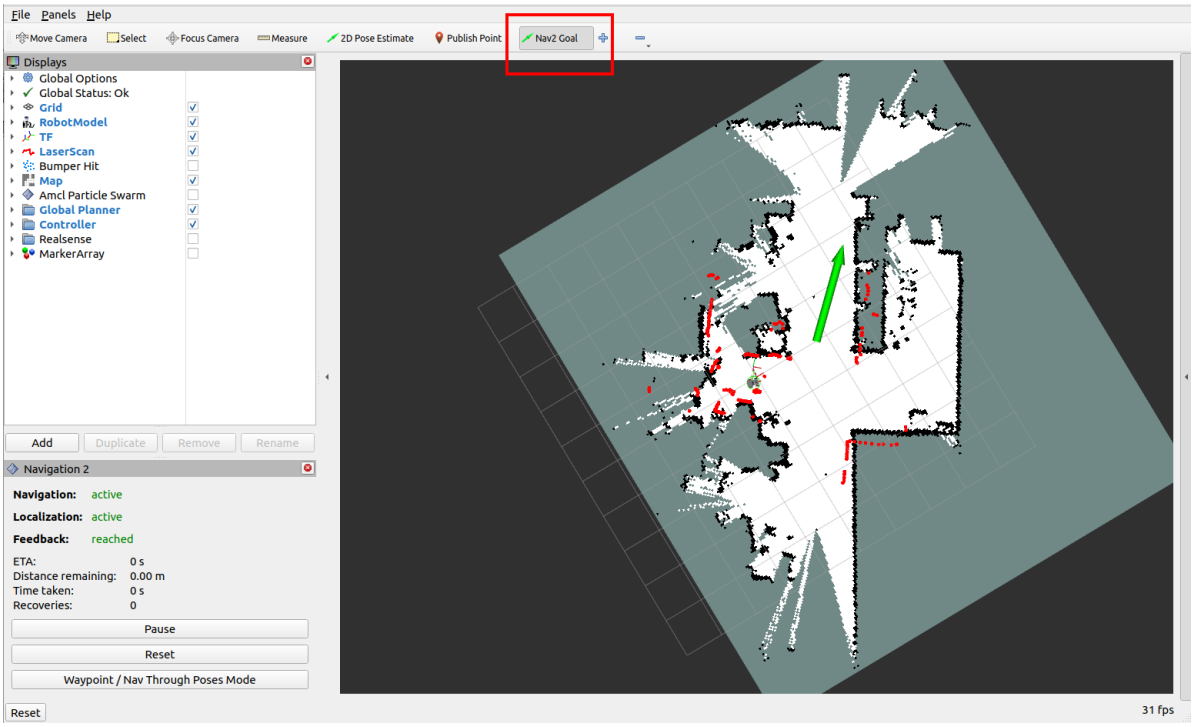
- At this point, you can see the map loaded. Then, click on "2D Pose Estimate" to set the initial pose of the car. Based on the car's actual position in the environment, click and drag with the mouse in RViz. The car model will move to the position we set. As shown in the figure below, if the Lidar scan area roughly coincides with the actual obstacles, the pose is accurate.



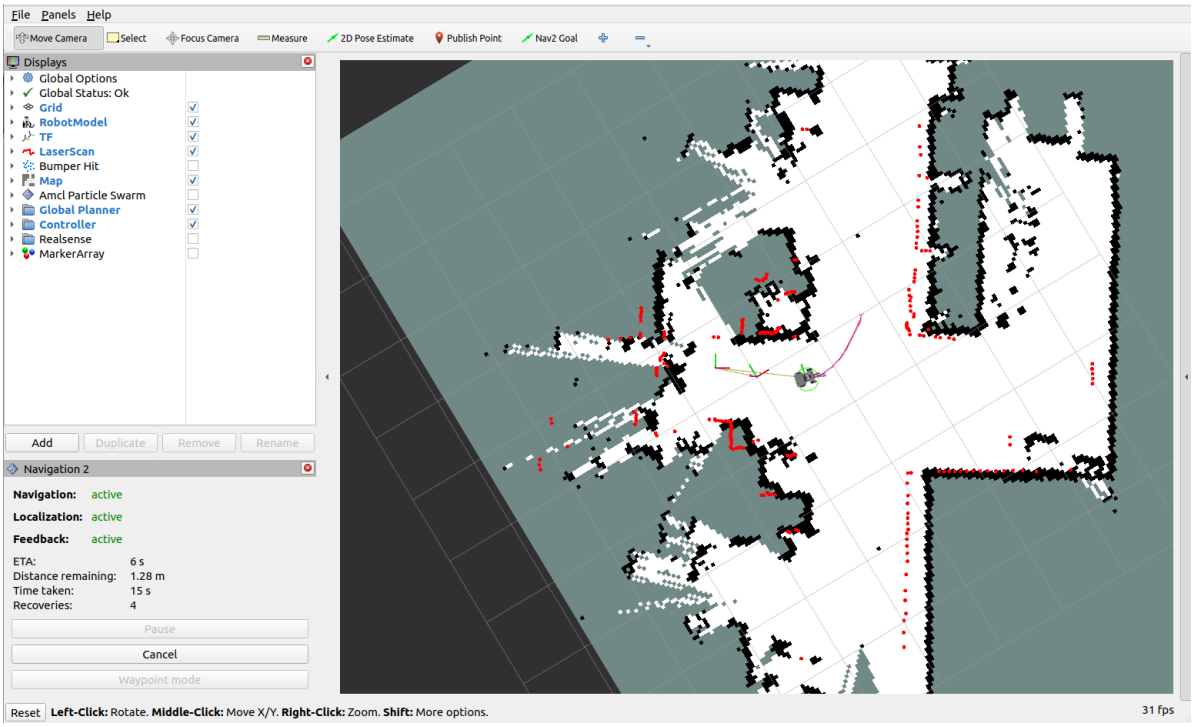
- After the pose is initialized, the robot model and the red 2D Lidar point cloud will appear in the RViz interface.



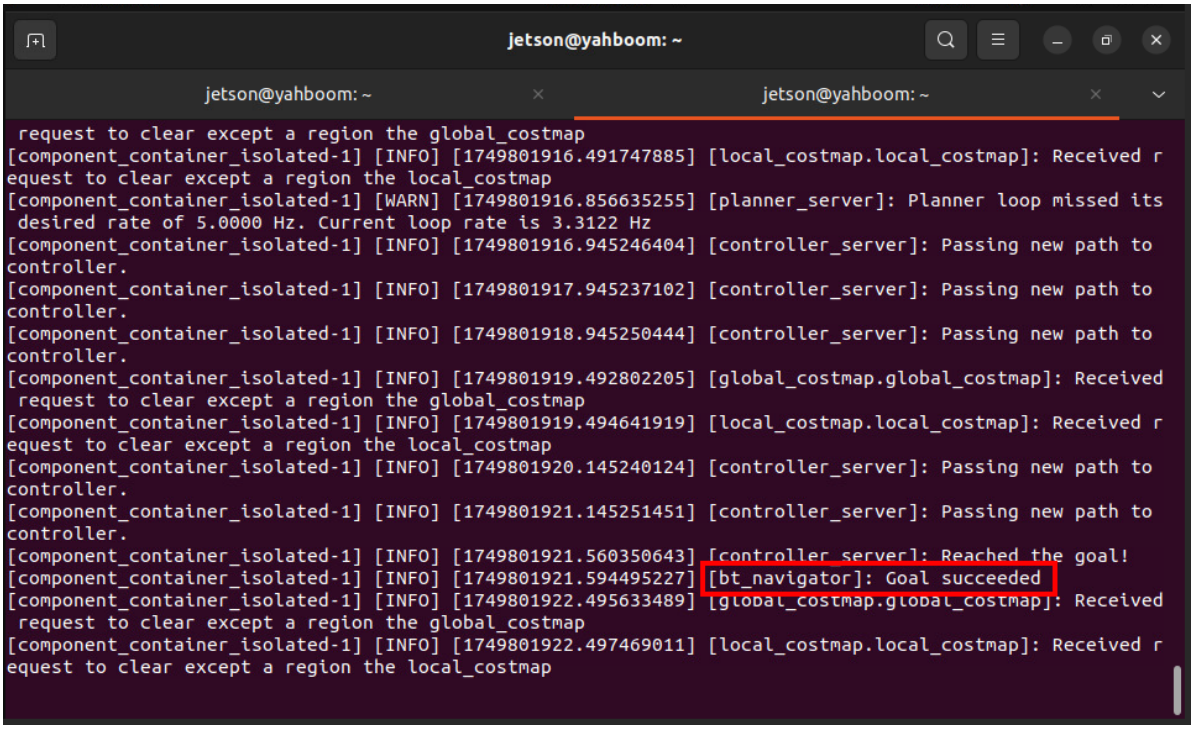
- For single-point navigation, click the "2D Goal Pose" tool, then use the mouse in RViz to select a target point and orientation, and then release.



- The car will plan a path based on its surroundings and move along the path to the target point.

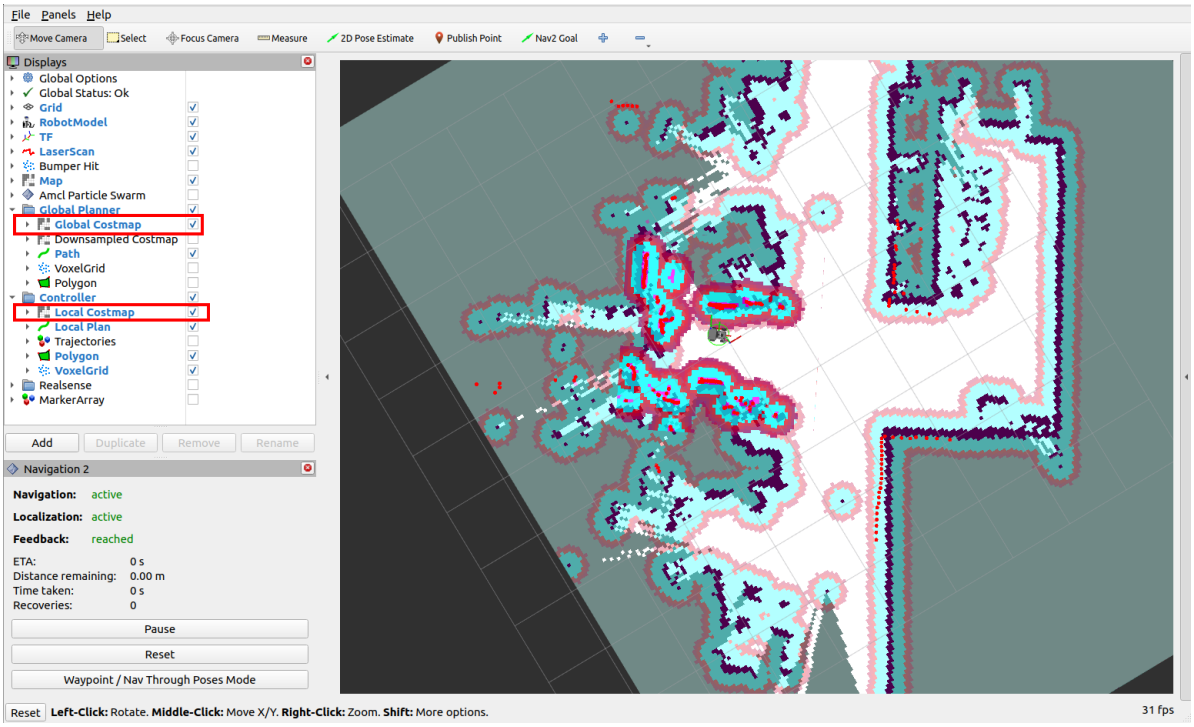


- After the robot successfully reaches the target point, the vehicle's terminal will display "Goal succeeded," indicating successful navigation.



4.2 View Costmap

- If you need to observe the global and local costmaps, find the `Global Costmap` under the `Global Planner` group in the left configuration panel and check the box to display the global costmap.
- Similarly, find the `Local Costmap` option under the `Controller` group and check the box to display the local costmap.



5. Navigation Explanation

5.1 Navigation Core Steps

5.1.1 Phase 1: System Initialization and Map Loading

Map Acquisition

- Load a pre-built grid map from `map_server`
- The map contains static obstacle information, serving as the basic environmental model for navigation.

Costmap Initialization

- **Global Costmap**: Based on the static map, used for global path planning.
- **Local Costmap**: Real-time fusion of Lidar's `scan` topic for dynamic obstacle avoidance.

5.1.2 Phase 2: Sensor Data Processing and Environment Perception

Sensor Data Input

- Lidar (`/scan`), IMU (`/imu/data`), and odometry (`/odom`) data are input through ROS topics.

Costmap Update

- The local costmap updates dynamic obstacle information in real-time, with each grid calculating occupancy probability and cost based on sensor data (e.g., the closer to an obstacle, the higher the cost).
- Inflation processing: Expand obstacle edges to prevent the robot from getting too close to obstacles.

5.1.3 Phase 3: Path Planning (Global and Local Planning)

Global Path Planning

- Input: Starting point (robot's current pose), end point (target pose), and global costmap.
- The planner algorithm Dijkstra generates the optimal path from start to end (a series of discrete coordinate points).

Local Path Planning and Tracking

The local planner DWBLocalPlanner generates short-term executable control commands for the robot based on the global path and local costmap.

5.1.4 Phase 4: Control Execution and Behavior Decision

Controller Output

- The controller converts the local path into the robot's linear velocity (`linear.x`) and angular velocity (`angular.z`), published through the `/cmd_vel` topic.
- Velocity smoothing: Avoid sudden robot movements and improve stability.

Behavior Tree Decision Flow

The behavior tree defines the priority and state transition logic of navigation tasks, with a typical flow:

1. **Check if target is reachable**: If global planning fails, trigger "recovery behavior" (e.g., replanning).
2. **Execute local obstacle avoidance**: When the local costmap detects obstacles, temporarily deviate from the global path.

3. **Reach target**: When the error between the robot's pose and target pose is less than a threshold, the task is complete.

Recovery Behavior Mechanism

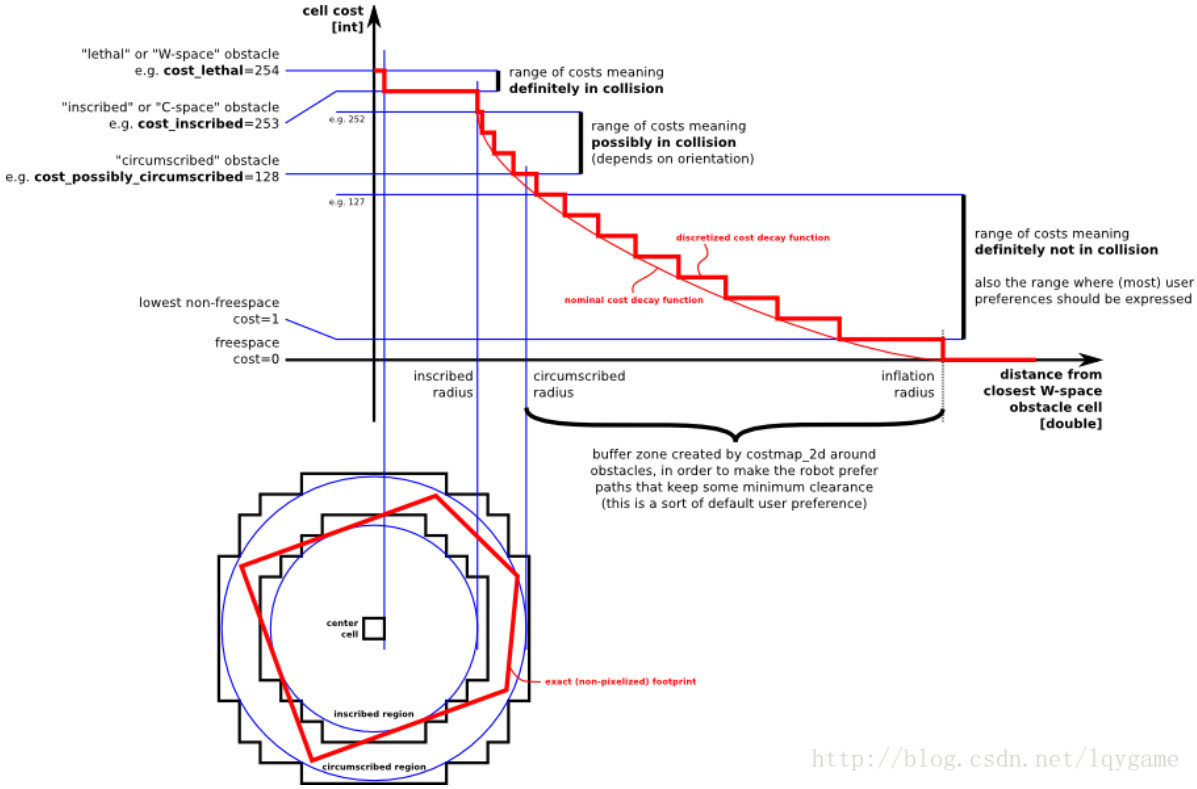
When navigation encounters exceptions (e.g., path is completely blocked), preset recovery strategies are triggered (e.g., rotate to search for a new path, back up and replan).

5.2 Key Technical Principles

5.2.1 Costmap

- **Probabilistic Grid Representation**: Each grid stores occupancy probability (0-1), updated through sensor data (e.g., Lidar ray detection).
- **Multi-layer Cost Superposition**: Static cost (map obstacles) + Dynamic cost (real-time sensor-detected obstacles) + Inflation cost (safety distance).

Costmap is a 2D or 3D map that the robot builds and updates by collecting sensor information, which can be briefly understood from the figure below.



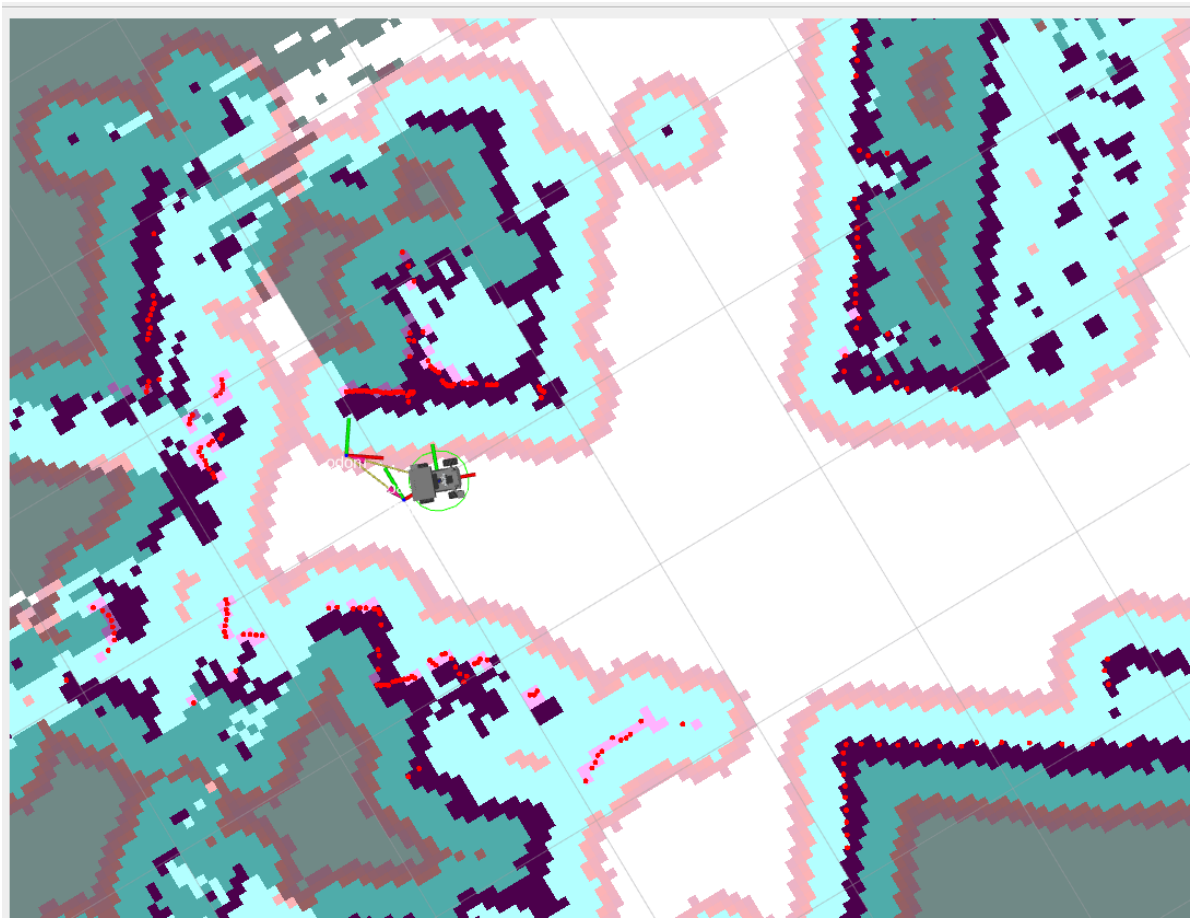
In the figure above, the red part represents obstacles in the costmap, the blue part represents obstacles inflated by the robot's inscribed circle radius, and the red polygon is the footprint (vertical projection of the robot's contour). To avoid collisions, the footprint should not intersect with the red part, and the robot's center should not intersect with the blue part.
ROS costmap uses a grid form, where each grid's value (cell cost) ranges from 0 to 255. It is divided into three states: occupied (with obstacles), free area (no obstacles), and unknown area; The specific states and corresponding values are shown in the figure below:

The figure above can be divided into five parts, where the red polygon area is the robot's contour:

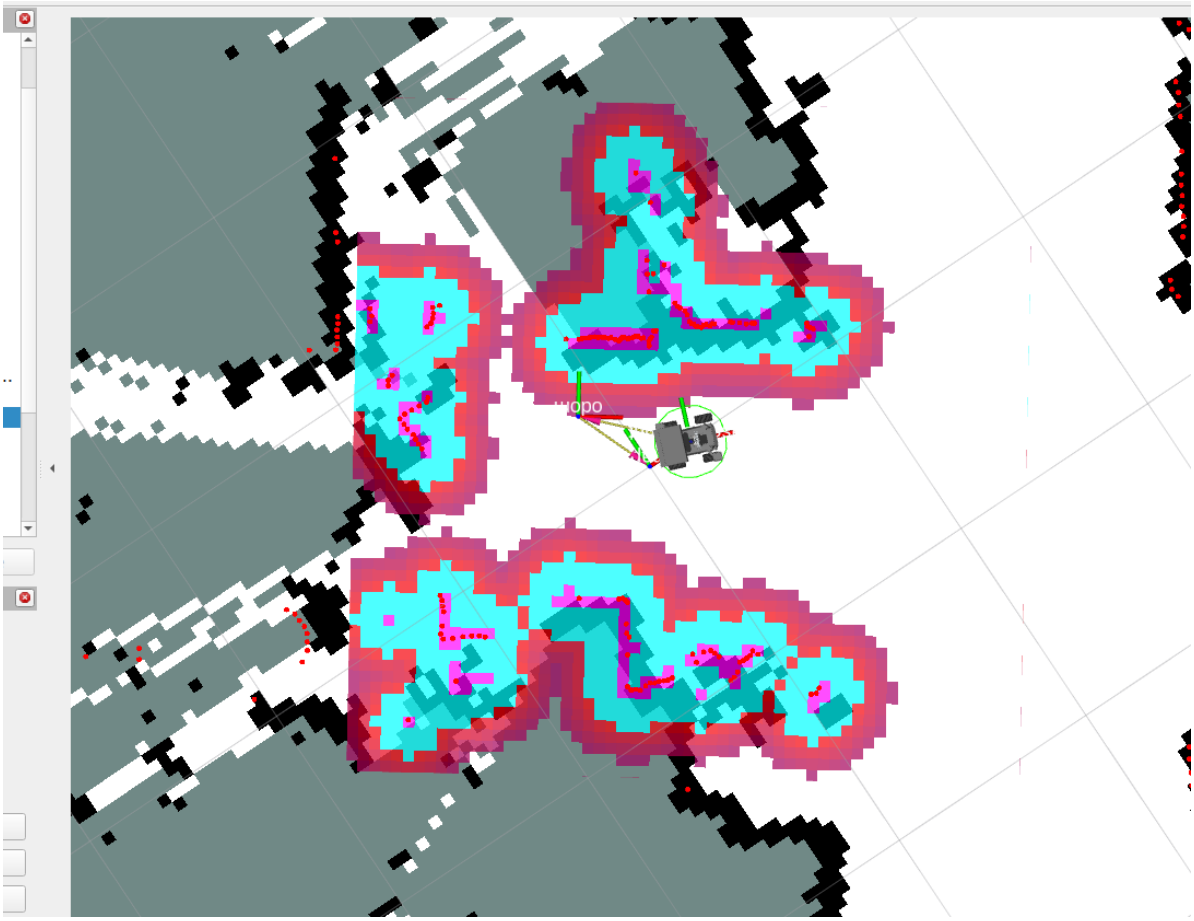
- Lethal: The robot's center coincides with the grid's center, in which case the robot will inevitably collide with obstacles.
- Inscribed: The grid's circumscribed circle is inscribed with the robot's contour, in which case the robot will also inevitably collide with obstacles.

- Possibly circumscribed: The grid's circumscribed circle is circumscribed with the robot's contour, in which case the robot is equivalent to being near obstacles, so collision is not certain.
- Freespace: Space without obstacles.
- Unknown: Unknown space.

Global Costmap:



Local Costmap:

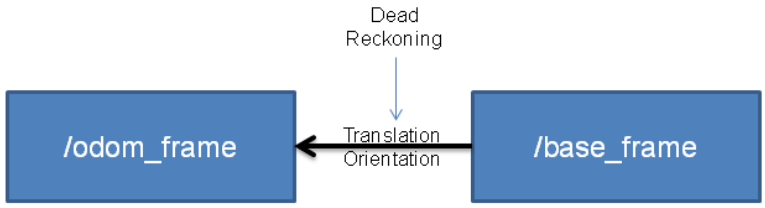


5.2.2 AMCL Localization Algorithm

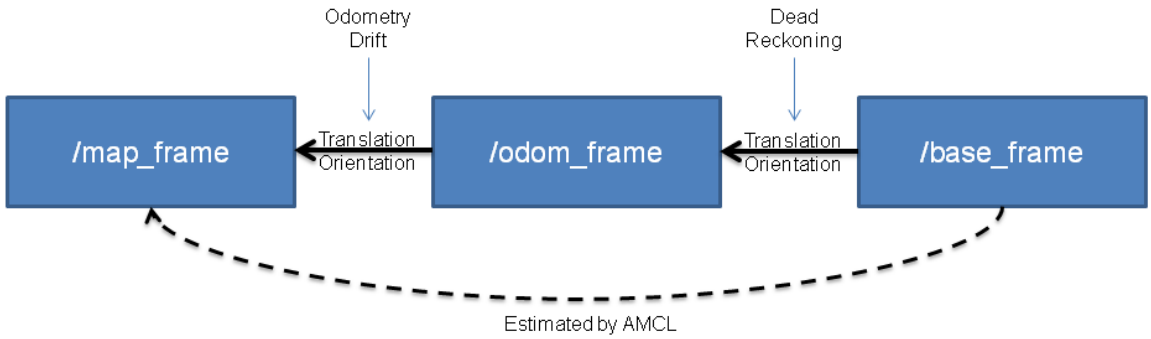
AMCL (Adaptive Monte Carlo Localization) is a probabilistic localization system for 2D mobile robots. It implements adaptive (or KLD sampling) Monte Carlo localization methods, which can use particle filters to calculate the robot's position based on existing maps.

As shown in the figure below, if odometry had no errors, in a perfect scenario, we could directly use odometry information (upper half of the figure) to calculate the robot's (base_frame) position relative to the odometry coordinate system. However, in reality, odometry has drift and non-negligible cumulative errors, so AMCL uses the method shown in the lower half of the figure: first, initially locate the base_frame based on odometry information, then obtain the base_frame relative to the map_frame (global map coordinate system) through the measurement model, thus knowing the robot's pose in the map. (Note: although this estimates the base-to-map transformation, what is finally published is the map-to-odom transformation, which can be understood as odometry drift.)

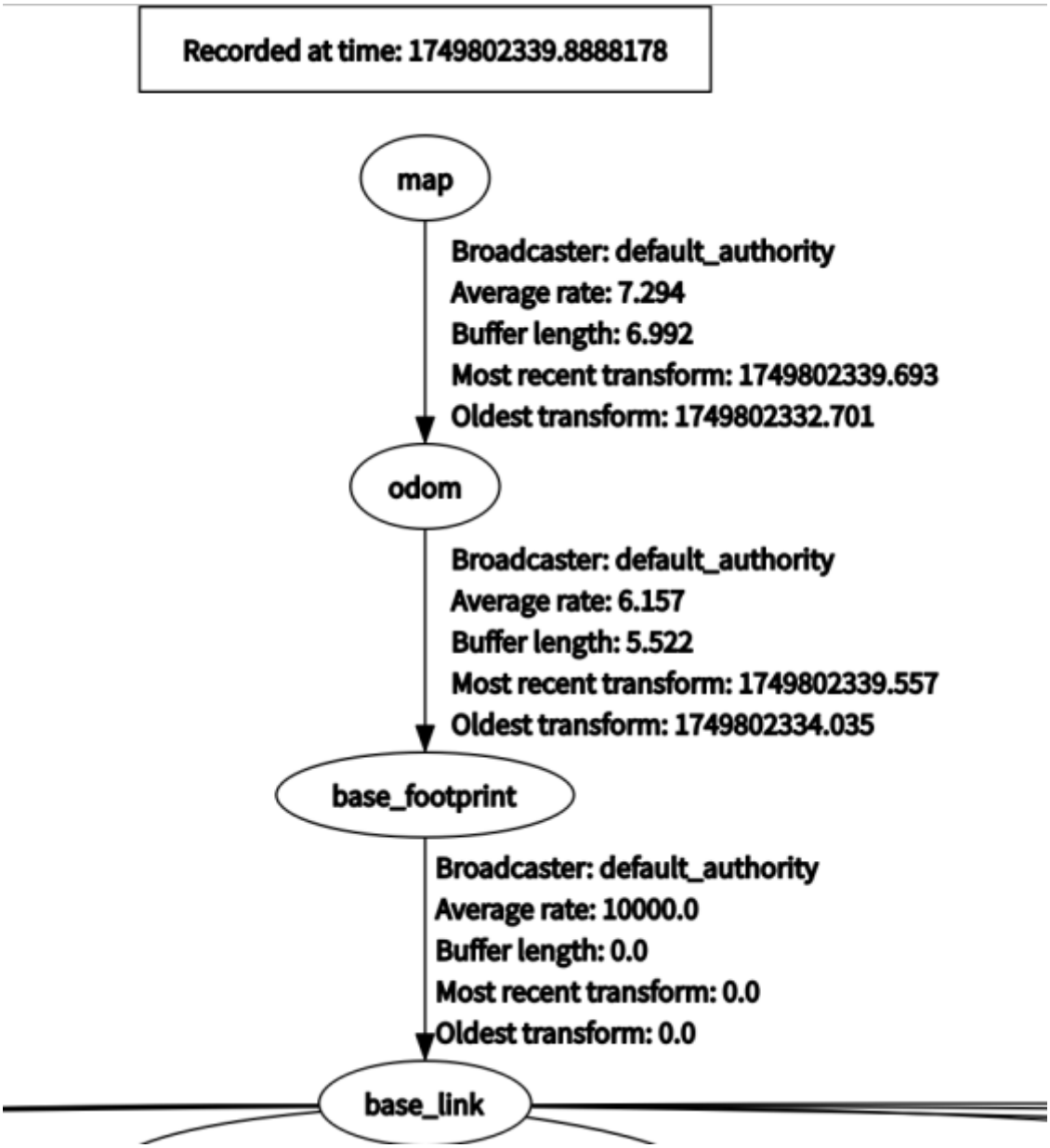
Odometry Localization



AMCL Map Localization



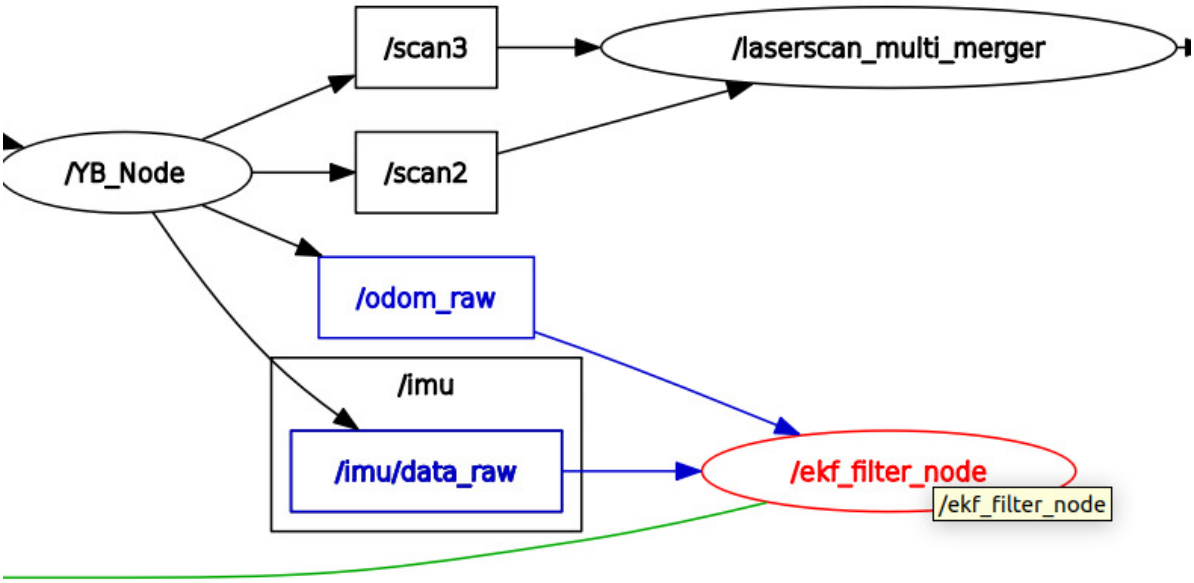
In the TF tree during robot navigation, the coordinate transformation between map (map coordinate system) and odom (odometry coordinate system) is published by the AMCL localization algorithm, while the coordinate transformation between odom (odometry coordinate system) and base_footprint (robot chassis center point projection coordinate system) is published by the **ekf_filter_node** node from the **robot_localization** package, which serves to publish odometry information after fusing IMU sensor and encoder raw odometry.



In the node communication diagram we viewed earlier, we can find the communication data flow of ekf_filter_node

Where the /YB_Node node is the robot's chassis node, publishing **/imu/data_raw** (IMU sensor raw data) and **/odom_raw** (encoder raw odometry data)

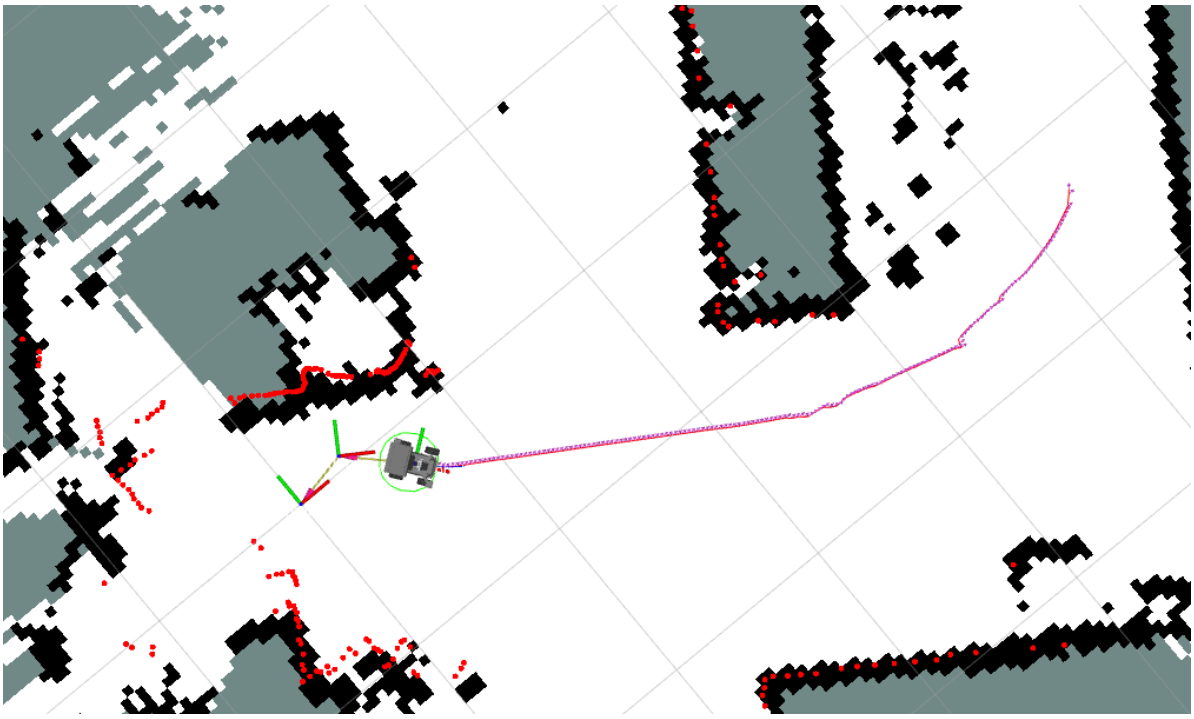
The **ekf_filter_node** node subscribes to the /**odom_raw** and **/imu/data_raw** topic data, fuses multi-sensor data and publishes the **/odom** topic



5.3 Path Planning

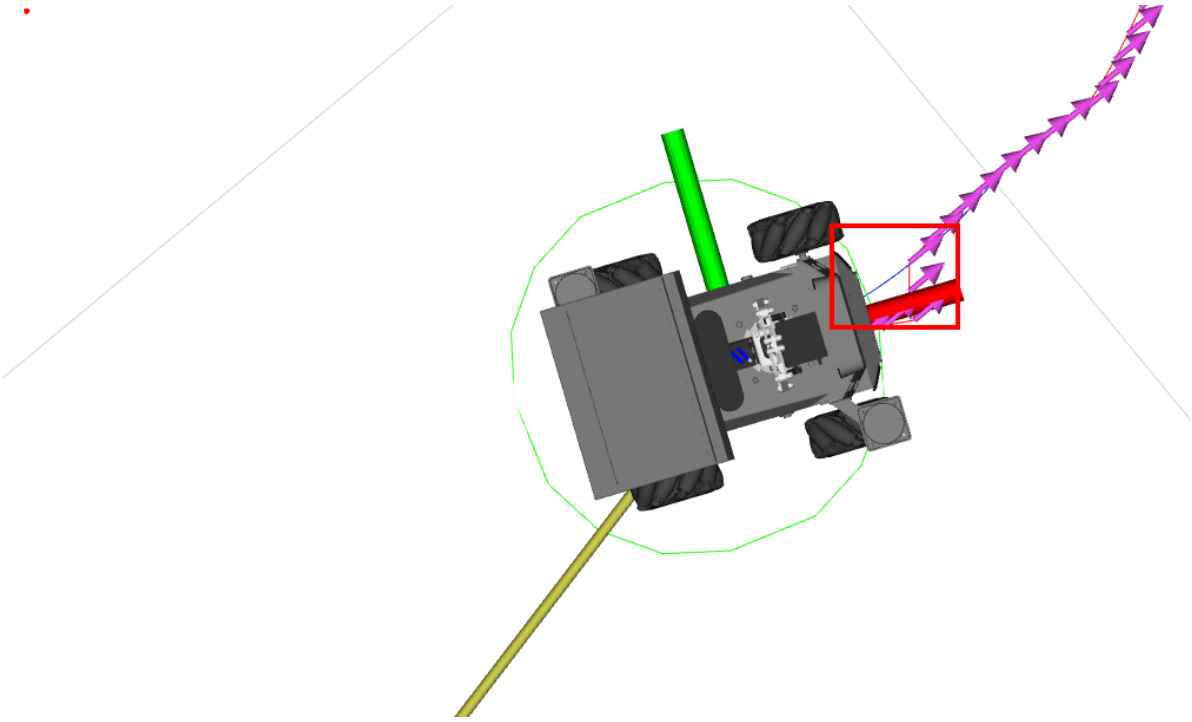
5.3.1 Global Path Planning

- The purpose of global path planning is to calculate an optimal or feasible global path in a known environment map based on the start and end points, without considering real-time obstacles, aiming for global optimality. It calculates the path from start to end as shown in the figure below, where the path formed by arrows is the global path.



5.3.2 Local Path Planning

- The purpose of local path planning is to dynamically adjust the path during robot movement based on real-time Lidar sensor data to avoid obstacles (obstacles will be displayed in the costmap). As shown in the figure below, the blue path is the local path.



6. Navigation Parameter Configuration

- Configuration file path:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/navigation_config/M3_nav2_params.yaml
```

6.1 Navigation Speed Tuning

Parameter Adjustment Guide

- If you need to adjust the movement speed and angular velocity during navigation, adjust `max_vel_x` and `max_vel_theta`
- If you need to adjust the accuracy of reaching the target point, adjust `xy_goal_tolerance` and `yaw_goal_tolerance`
- For other parameter adjustments, see the comments below

6.2 Obstacle Avoidance Effect Tuning

Parameter Adjustment Guide

- If the robot gets too close to obstacles in the map during navigation, you can make the robot stay away from obstacles by adjusting the inflation area. Increase the `inflation_radius` (inflation radius) and `cost_scaling_factor` (scaling factor; the larger the value, the less likely the robot will approach obstacles) in `local_costmap` and `global_costmap`
- For other parameter adjustments, see the comments below

```yaml
# AMCL (Adaptive Monte Carlo Localization) Configuration
amcl:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False
    # Noise parameter for odometry model when robot rotates
    alpha1: 0.2
    # Noise parameter for odometry model when robot rotation causes translation
    alpha2: 0.2
    # Noise parameter for odometry model when robot translates
    alpha3: 0.2
    # Noise parameter for odometry model when robot translation causes rotation
    alpha4: 0.2
    # Noise parameter for odometry model when robot rotates (another aspect)
    alpha5: 0.2
    # Robot's base coordinate frame name
    base_frame_id: "base_footprint"
    # Distance threshold for beam skipping, used for laser scan data processing
    beam_skip_distance: 0.5
    # Error threshold for beam skipping, used to decide whether to skip certain
beams
    beam_skip_error_threshold: 0.9
    # Threshold for beam skipping, used to decide whether to skip certain beams
    beam_skip_threshold: 0.3
    # Whether to enable beam skipping function
    do_beamskip: false
    # Global coordinate frame name
    global_frame_id: "map"
    # Decay factor for short laser reflections
    lambda_short: 0.1
    # Maximum likelihood distance for laser scan
    laser_likelihood_max_dist: 2.0
    # Maximum range of laser scan
    laser_max_range: 100.0
    # Minimum range of laser scan
    laser_min_range: -1.0
    # Laser model type
    laser_model_type: "likelihood_field"
    # Maximum number of laser beams used per update
    max_beams: 60
    # Maximum number of particles allowed in particle filter
    max_particles: 2000
    # Minimum number of particles allowed in particle filter
    min_particles: 500
    # Odometry coordinate frame name
    odom_frame_id: "odom"
    # Error threshold of particle filter
    pf_err: 0.05
    # Confidence threshold of particle filter
    pf_z: 0.99
    # Fast recovery factor, used for particle filter recovery mechanism
    recovery_alpha_fast: 0.0
    # Slow recovery factor, used for particle filter recovery mechanism
    recovery_alpha_slow: 0.0
    # Resampling interval
    resample_interval: 1
    # Robot's motion model type
```

```yaml
    robot_model_type: "nav2_amcl::DifferentialMotionModel"
    # Rate at which to save robot pose
    save_pose_rate: 0.5
    # Standard deviation of laser hit
    sigma_hit: 0.2
    # Whether to broadcast TF transform
    tf_broadcast: true
    # TF transform publish rate
    tf_publish_rate: 10.0
    # Transform tolerance time
    transform_tolerance: 0.2
    # Minimum threshold for angle update
    update_min_a: 0.2
    # Minimum threshold for distance update
    update_min_d: 0.25
    # Weight of laser hit
    z_hit: 0.5
    # Weight of laser max range
    z_max: 0.05
    # Weight of random laser measurement
    z_rand: 0.5
    # Weight of short laser reflection
    z_short: 0.05
    # Topic name of laser scan data
    scan_topic: scan

# AMCL Map Client Configuration
amcl_map_client:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False

# AMCL RCLCPP Node Configuration
amcl_rclcpp_node:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False

# BT Navigator Configuration
bt_navigator:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False
    # Global coordinate frame name
    global_frame: map
    # Robot's base coordinate frame name
    robot_base_frame: base_link
    # Topic name of odometry data
    odom_topic: /odom
    # Default behavior tree XML file name
    default_bt_xml_filename: "navigate_w_replanning_and_recovery.xml"
    # Behavior tree loop duration
    bt_loop_duration: 10
    # Default server timeout
    default_server_timeout: 20
    # Whether to enable Groot monitoring
    enable_groot_monitoring: True
    # Groot's ZMQ publisher port
```

```yaml
    groot_zmq_publisher_port: 1666
    # Groot's ZMQ server port
    groot_zmq_server_port: 1667
    # List of behavior tree plugin library names
    plugin_lib_names:
    - nav2_compute_path_to_pose_action_bt_node
    - nav2_compute_path_through_poses_action_bt_node
    - nav2_follow_path_action_bt_node
    - nav2_back_up_action_bt_node
    - nav2_spin_action_bt_node
    - nav2_wait_action_bt_node
    - nav2_clear_costmap_service_bt_node
    - nav2_is_stuck_condition_bt_node
    - nav2_goal_reached_condition_bt_node
    - nav2_goal_updated_condition_bt_node
    - nav2_initial_pose_received_condition_bt_node
    - nav2_reinitialize_global_localization_service_bt_node
    - nav2_rate_controller_bt_node
    - nav2_distance_controller_bt_node
    - nav2_speed_controller_bt_node
    - nav2_truncate_path_action_bt_node
    - nav2_goal_updater_node_bt_node
    - nav2_recovery_node_bt_node
    - nav2_pipeline_sequence_bt_node
    - nav2_round_robin_node_bt_node
    - nav2_transform_available_condition_bt_node
    - nav2_time_expired_condition_bt_node
    - nav2_distance_traveled_condition_bt_node
    - nav2_single_trigger_bt_node
    - nav2_is_battery_low_condition_bt_node
    - nav2_navigate_through_poses_action_bt_node
    - nav2_navigate_to_pose_action_bt_node
    - nav2_remove_passed_goals_action_bt_node
    - nav2_planner_selector_bt_node
    - nav2_controller_selector_bt_node
    - nav2_goal_checker_selector_bt_node

# BT Navigator RCLCPP Node Configuration
bt_navigator_rclcpp_node:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False

# Controller Server Configuration
controller_server:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False
    # Controller operating frequency
    controller_frequency: 5.0
    # Minimum velocity threshold in the X direction
    min_x_velocity_threshold: 0.001
    # Minimum velocity threshold in the Y direction
    min_y_velocity_threshold: 0.5
    # Minimum rotational velocity threshold
    min_theta_velocity_threshold: 0.001
    # Maximum allowed failure tolerance
    failure_tolerance: 0.3
```

```yaml
    # Progress checker plugin name
    progress_checker_plugin: "progress_checker"
    # List of goal checker plugin names
    goal_checker_plugins: ["general_goal_checker"]
    # List of controller plugin names
    controller_plugins: ["FollowPath"]

    # Progress checker parameters
    progress_checker:
      # Progress checker plugin type
      plugin: "nav2_controller::SimpleProgressChecker"
      # Minimum radius the robot must move
      required_movement_radius: 0.5
      # Maximum allowed movement time
      movement_time_allowance: 10.0

    general_goal_checker:
      # Whether it is a stateful goal checker
      stateful: True
      # Goal checker plugin type
      plugin: "nav2_controller::SimpleGoalChecker"
      # Goal tolerance in the X and Y directions
      xy_goal_tolerance: 0.15
      # Goal tolerance in the yaw direction
      yaw_goal_tolerance: 0.15

    # DWB Local Planner Parameters
    FollowPath:
      # Local planner plugin type
      plugin: "dwb_core::DWBLocalPlanner"
      # Whether to debug trajectory details
      debug_trajectory_details: True
      # Minimum velocity in the X direction
      min_vel_x: 0.0
      # Minimum velocity in the Y direction
      min_vel_y: 0.0
      # Maximum velocity in the X direction
      max_vel_x: 0.35
      # Maximum velocity in the Y direction
      max_vel_y: 0.0
      # Maximum rotational velocity
      max_vel_theta: 1.0
      # Minimum speed in the XY plane
      min_speed_xy: 0.0
      # Maximum speed in the XY plane
      max_speed_xy: 0.22
      # Minimum rotational speed
      min_speed_theta: 0.0
      # Acceleration limit in the X direction
      acc_lim_x: 2.5
      # Acceleration limit in the Y direction
      acc_lim_y: 0.0
      # Rotational acceleration limit
      acc_lim_theta: 3.2
      # Deceleration limit in the X direction
      decel_lim_x: -2.5
      # Deceleration limit in the Y direction
      decel_lim_y: 0.0
```

```
      # Rotational deceleration limit
      decel_lim_theta: -3.2
      # Number of velocity samples in the X direction
      vx_samples: 20
      # Number of velocity samples in the Y direction
      vy_samples: 0
      # Number of rotational velocity samples
      vtheta_samples: 40
      # Simulation time
      sim_time: 1.5
      # Linear granularity
      linear_granularity: 0.05
      # Angular granularity
      angular_granularity: 0.025
      # Transform tolerance time
      transform_tolerance: 0.2
      # Goal tolerance in the X and Y directions
      xy_goal_tolerance: 0.05
      # Velocity threshold for translational stop
      trans_stopped_velocity: 0.25
      # Whether to short-circuit trajectory evaluation
      short_circuit_trajectory_evaluation: True
      # Whether it is a stateful planner
      stateful: True
      # List of critics
      critics: ["RotateToGoal", "Oscillation", "BaseObstacle", "GoalAlign",
"PathAlign", "PathDist", "GoalDist"]
      # Weight of the base obstacle critic
      BaseObstacle.scale: 0.02
      # Weight of the path alignment critic
      PathAlign.scale: 32.0
      # Forward point distance for the path alignment critic
      PathAlign.forward_point_distance: 0.1
      # Weight of the goal alignment critic
      GoalAlign.scale: 24.0
      # Forward point distance for the goal alignment critic
      GoalAlign.forward_point_distance: 0.1
      # Weight of the path distance critic
      PathDist.scale: 32.0
      # Weight of the goal distance critic
      GoalDist.scale: 24.0
      # Weight of the rotate-to-goal critic
      RotateToGoal.scale: 32.0
      # Slowing factor for the rotate-to-goal critic
      RotateToGoal.slowing_factor: 5.0
      # Lookahead time for the rotate-to-goal critic
      RotateToGoal.lookahead_time: -1.0

# Controller Server RCLCPP Node Configuration
controller_server_rclcpp_node:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False

# Local Costmap Configuration
local_costmap:
  local_costmap:
    ros__parameters:
```

```yaml
    # Costmap update frequency
    update_frequency: 5.0
    # Costmap publish frequency
    publish_frequency: 2.0
    # Global coordinate frame name
    global_frame: odom
    # Robot's base coordinate frame name
    robot_base_frame: base_link
    # Whether to use simulation time, False means use actual time
    use_sim_time: False
    # Whether to use a rolling window
    rolling_window: true
    # Costmap width
    width: 3
    # Costmap height
    height: 3
    # Costmap resolution
    resolution: 0.05
    # Robot's radius
    robot_radius: 0.15
    # List of costmap plugins
    plugins: ["obstacle_layer", "voxel_layer", "inflation_layer"]
    inflation_layer:
      # Inflation layer plugin type
      plugin: "nav2_costmap_2d::InflationLayer"
      # Inflation radius
      inflation_radius: 0.3
      # Cost scaling factor
      cost_scaling_factor: 3.0
    obstacle_layer:
      # Obstacle layer plugin type
      plugin: "nav2_costmap_2d::ObstacleLayer"
      # Whether to enable the obstacle layer
      enabled: True
      # Observation source name
      observation_sources: scan
      scan:
        # Topic name for laser scan data
        topic: /scan
        # Maximum obstacle height
        max_obstacle_height: 2.0
        # Whether to clear obstacles
        clearing: True
        # Whether to mark obstacles
        marking: True
        # Data type
        data_type: "LaserScan"
    voxel_layer:
      # Voxel layer plugin type
      plugin: "nav2_costmap_2d::VoxelLayer"
      # Whether to enable the voxel layer
      enabled: True
      # Whether to publish the voxel map
      publish_voxel_map: True
      # Origin in the Z direction
      origin_z: 0.0
      # Resolution in the Z direction
      z_resolution: 0.05
```

```yaml
        # Number of voxels in the Z direction
        z_voxels: 16
        # Maximum obstacle height
        max_obstacle_height: 2.0
        # Mark threshold
        mark_threshold: 0
        # Observation source name
        observation_sources: scan
        scan:
          # Topic name for laser scan data
          topic: /scan
          # Maximum obstacle height
          max_obstacle_height: 2.0
          # Whether to clear obstacles
          clearing: True
          # Whether to mark obstacles
          marking: True
          # Data type
          data_type: "LaserScan"
          # Maximum range for raytracing
          raytrace_max_range: 3.0
          # Minimum range for raytracing
          raytrace_min_range: 0.0
          # Maximum detection range for obstacles
          obstacle_max_range: 2.5
          # Minimum detection range for obstacles
          obstacle_min_range: 0.0
      static_layer:
        # Whether to subscribe to the transient local map
        map_subscribe_transient_local: True
      # Whether to always send the full costmap
      always_send_full_costmap: True
  local_costmap_client:
    ros__parameters:
      # Whether to use simulation time, False means use actual time
      use_sim_time: False
  local_costmap_rclcpp_node:
    ros__parameters:
      # Whether to use simulation time, False means use actual time
      use_sim_time: False

# Global Costmap Configuration
global_costmap:
  global_costmap:
    ros__parameters:
      # Costmap update frequency
      update_frequency: 1.0
      # Costmap publish frequency
      publish_frequency: 1.0
      # Global coordinate frame name
      global_frame: map
      # Robot's base coordinate frame name
      robot_base_frame: base_link
      # Whether to use simulation time, True means use simulation time
      use_sim_time: True
      # Robot's radius
      robot_radius: 0.2
      # Costmap resolution
```

```
      resolution: 0.05
      # Whether to track unknown space
      track_unknown_space: false
      # List of costmap plugins
      plugins: ["static_layer", "obstacle_layer", "voxel_layer",
"inflation_layer"]
      obstacle_layer:
        # Obstacle layer plugin type
        plugin: "nav2_costmap_2d::ObstacleLayer"
        # Whether to enable the obstacle layer
        enabled: True
        # Observation source name
        observation_sources: scan
        scan:
          # Topic name for laser scan data
          topic: /scan
          # Maximum obstacle height
          max_obstacle_height: 2.0
          # Whether to clear obstacles
          clearing: True
          # Whether to mark obstacles
          marking: True
          # Data type
          data_type: "LaserScan"
          # Maximum range for raytracing
          raytrace_max_range: 3.0
          # Minimum range for raytracing
          raytrace_min_range: 0.0
          # Maximum detection range for obstacles
          obstacle_max_range: 2.5
          # Minimum detection range for obstacles
          obstacle_min_range: 0.0
      voxel_layer:
        # Voxel layer plugin type
        plugin: "nav2_costmap_2d::VoxelLayer"
        # Whether to enable the voxel layer
        enabled: True
        # Whether to publish the voxel map
        publish_voxel_map: True
        # Origin in the Z direction
        origin_z: 0.0
        # Resolution in the Z direction
        z_resolution: 0.05
        # Number of voxels in the Z direction
        z_voxels: 16
        # Maximum obstacle height
        max_obstacle_height: 2.0
        # Mark threshold
        mark_threshold: 0
        # Observation source name
        observation_sources: scan
        scan:
          # Topic name for laser scan data
          topic: /scan
          # Maximum obstacle height
          max_obstacle_height: 2.0
          # Whether to clear obstacles
          clearing: True
```

```yaml
          # Whether to mark obstacles
          marking: True
          # Data type
          data_type: "LaserScan"
          # Maximum range for raytracing
          raytrace_max_range: 3.0
          # Minimum range for raytracing
          raytrace_min_range: 0.0
          # Maximum detection range for obstacles
          obstacle_max_range: 2.5
          # Minimum detection range for obstacles
          obstacle_min_range: 0.0
      static_layer:
        # Static layer plugin type
        plugin: "nav2_costmap_2d::StaticLayer"
        # Whether to subscribe to the transient local map
        map_subscribe_transient_local: True
      inflation_layer:
        # Inflation layer plugin type
        plugin: "nav2_costmap_2d::InflationLayer"
        # Cost scaling factor
        cost_scaling_factor: 3.0
        # Inflation radius
        inflation_radius: 0.3
      # Whether to always send the full costmap
      always_send_full_costmap: True
  global_costmap_client:
    ros__parameters:
      # Whether to use simulation time, False means use actual time
      use_sim_time: False
  global_costmap_rclcpp_node:
    ros__parameters:
      # Whether to use simulation time, False means use actual time
      use_sim_time: False

# Map Server Configuration
map_server:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False
    # Name of the map YAML file
    yaml_filename: "map.yaml"

# Map Saver Configuration
map_saver:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False
    # Timeout for saving the map
    save_map_timeout: 5.0
    # Default free threshold
    free_thresh_default: 0.25
    # Default occupied threshold
    occupied_thresh_default: 0.65
    # Whether to subscribe to the transient local map
    map_subscribe_transient_local: True

# Planner Server Configuration
```

```yaml
planner_server:
  ros__parameters:
    # Expected planner frequency
    expected_planner_frequency: 5.0
    # Whether to use simulation time, False means use actual time
    use_sim_time: False
    # List of planner plugin names
    planner_plugins: ["GridBased"]
    GridBased:
      # Grid-based planner plugin type
      plugin: "nav2_navfn_planner/NavfnPlanner"
      # Planning tolerance
      tolerance: 0.5
      # Whether to use A* algorithm
      use_astar: false
      # Whether to allow unknown space
      allow_unknown: true

# Planner Server RCLCPP Node Configuration
planner_server_rclcpp_node:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False

# Recoveries Server Configuration
recoveries_server:
  ros__parameters:
    # Topic name for the costmap
    costmap_topic: local_costmap/costmap_raw
    # Topic name for the robot footprint
    footprint_topic: local_costmap/published_footprint
    # Cycle frequency
    cycle_frequency: 5.0
    # List of recovery plugin names
    recovery_plugins: ["spin", "backup", "wait"]
    spin:
      # Spin recovery plugin type
      plugin: "nav2_recoveries/Spin"
    backup:
      # Backup recovery plugin type
      plugin: "nav2_recoveries/BackUp"
    wait:
      # Wait recovery plugin type
      plugin: "nav2_recoveries/Wait"
    # Global coordinate frame name
    global_frame: odom
    # Robot's base coordinate frame name
    robot_base_frame: base_link
    # Transform timeout
    transform_timeout: 0.1
    # Whether to use simulation time, False means use actual time
    use_sim_time: False
    # Lookahead simulation time
    simulate_ahead_time: 2.0
    # Maximum rotational velocity
    max_rotational_vel: 1.0
    # Minimum rotational velocity
    min_rotational_vel: 0.4
```

```yaml
    # Rotational acceleration limit
    rotational_acc_lim: 3.2

# Robot State Publisher Configuration
robot_state_publisher:
  ros__parameters:
    # Whether to use simulation time, False means use actual time
    use_sim_time: False

# Waypoint Follower Configuration
waypoint_follower:
  ros__parameters:
    # Loop rate
    loop_rate: 2000
    # Whether to stop on failure
    stop_on_failure: false
    # Waypoint task executor plugin name
    waypoint_task_executor_plugin: "wait_at_waypoint"
    wait_at_waypoint:
      # Waypoint wait plugin type
      plugin: "nav2_waypoint_follower::WaitAtWaypoint"
      # Whether to enable the plugin
      enabled: True
      # Waypoint pause duration
      waypoint_pause_duration: 200
```